



Lecture 4 : Transformation (génération de code) par programmation

Ingénierie Dirigée par les Modèles (IDM / MDE) – version 2027

Du modèle Ecore `Library` au code Python, SQL et Java avec l'API EMF

Pr. Aissam BERRAHOU

Département Ingénierie des systèmes embarqués et mobiles – Smart Systems Lab
ENSIAS, Université Mohammed V de Rabat

aissam.berrahou@ensias.um5.ac.ma

Feuille de route

- **Pourquoi programmer ?** : *Qu'est-ce qu'une transformation par programmation et quand l'utiliser ?*
- **Lire un modèle avec EMF** : *Comment charger et parcourir un modèle Ecore en Java ?*
- **Le générateur pas à pas** : *Comment écrire, étape par étape, un programme qui génère du code ?*
- **Cas complet : Python, SQL, Java** : *Comment générer Python, SQL et Java à partir de Library.ecore ?*
- **Bonnes pratiques et synthèse** : *Quelles erreurs éviter ? Programmation ou template ? Que retenir ?*

Objectifs de la séance

À la fin de cette séance, vous saurez :

- Expliquer ce qu'est une transformation **par programmation** et la situer par rapport à **ATL** (M2M) et **Acceleo** (M2T par template).
- **Créer** un projet Java qui utilise l'**API EMF** (bibliothèques `org.eclipse.emf.*`).
- **Charger** un fichier `.ecore` et le **parcourir** : `getEClassifiers()`, `getEAttributes()`, `getEReferences()`...
- **Écrire** un générateur : boucles, conditions, méthodes utilitaires, écriture de fichiers.
- **Générer** à partir de `Library.ecore` des classes **Python**, un script **SQL** et des classes **Java**.

> **!** Lecture 1 (modèle, méta-modèle, Ecore), Lecture 3 (Acceleo) et bases de **Java** (classes, boucles `for`, listes).

Pourquoi programmer ?

Qu'est-ce qu'une transformation par programmation et quand l'utiliser ?

Rappel : trois façons de transformer un modèle

→ Ce que nous avons vu... et ce que nous voyons aujourd'hui

M2M avec un langage dédié

ATL, **QVT** : des **règles** déclaratives transforment un modèle en un **autre modèle**.

Lecture 2.

M2T par template

Acceleo : un **texte à trous** est rempli avec les données du modèle.

Lecture 3.

Par programmation

Un **programme** (Java, Python...) lit le modèle grâce à une **API** et produit ce qu'il veut : texte **ou** modèle.

Aujourd'hui.



Transformer par programmation, c'est écrire **soi-même** le programme qui **lit** le modèle et **écrit** le résultat, avec un langage généraliste.

Définition

→ La transformation par programmation en une image

Transformation par Programmation en MDA

Définition: Créer des transformations entre modèles en utilisant des langages de programmation (ex: Java, Python)



Exemples:

- Ecore → Java
- Ecore → Python
- Ecore → Python
- Ecore → SQL

Outils de Transformation:

- EMF (Eclipse Modeling Framework)
- PyEcore
- Manuel

- **Entrée** : un modèle conforme à un méta-modèle (ici un fichier `.ecore`).
- **Programme** : écrit en Java (API **EMF**) ou en Python (**PyEcore**).
- **Sortie** : du code Java, Python, SQL. . . (ou un autre modèle).

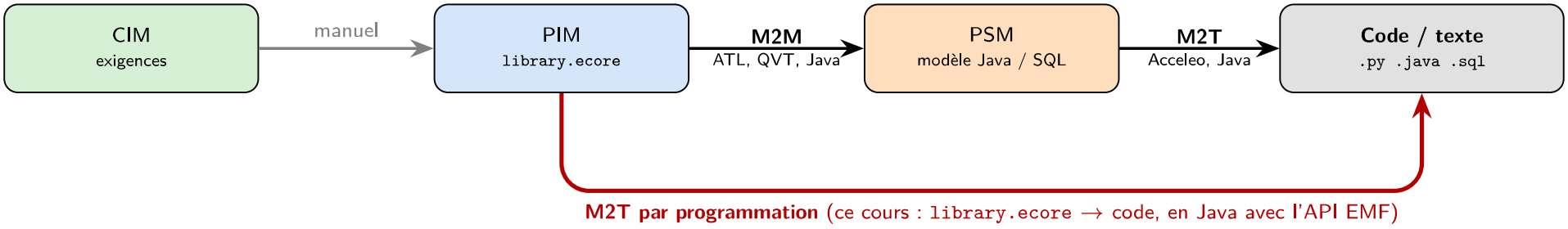
> **!** Le programme est **impératif** : on dit **pas à pas** comment parcourir le modèle et quoi écrire.

Où se situe-t-elle dans MDA ?

→ Une approche générale, utilisable à chaque marche

CIM | PIM | PSM | **Code**

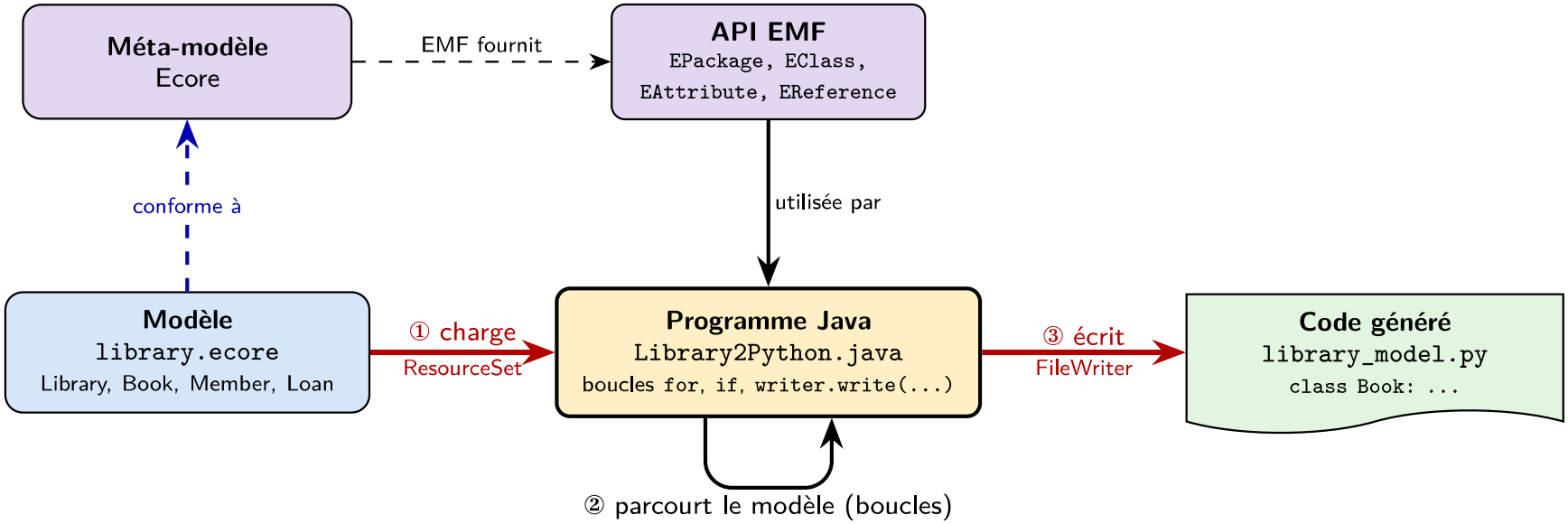
*La programmation est l'approche la plus **générale** : elle sert aussi bien au M2M qu'au M2T*



- En **M2M**, le programme lit un modèle et **crée** un autre modèle (méthodes `create...()` d'EMF).
- En **M2T** (ce cours), le programme lit un modèle et **écrit du texte** : du code source.
- Comme dans le tutoriel, on génère **directement** le code à partir de `library.ecore`.

Le principe : charger, parcourir, écrire

→ Trois étapes, toujours les mêmes



- ① **Charger** le fichier `.ecore` en mémoire (objets Java) avec un `ResourceSet`.
- ② **Parcourir** le modèle avec des boucles `for` et les **getters** de l'API EMF.
- ③ **Écrire** le texte produit dans un fichier avec un `FileWriter`.

Une analogie : le secrétaire

→ Dicter mot à mot ou donner une lettre type ?

Par template (Acceleo)

On donne au secrétaire une **lettre type à trous** et la liste des destinataires. Il remplit les trous.

> ! On **voit** directement le résultat final dans la lettre type.

Par programmation (Java)

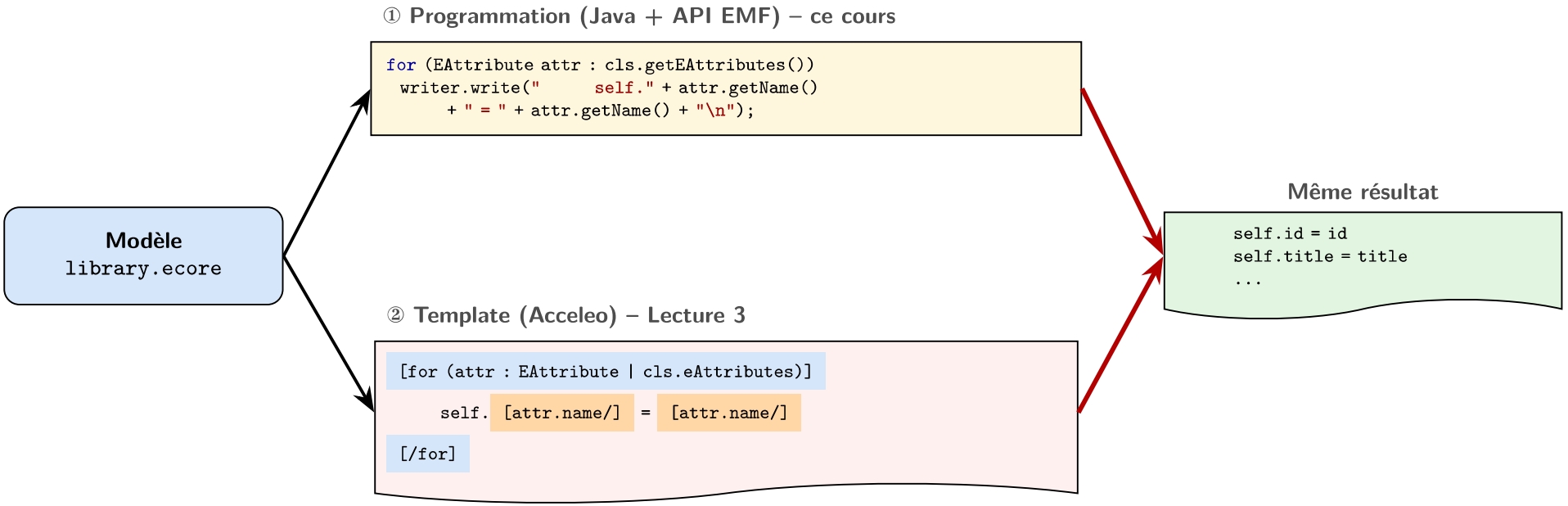
On **dicte** chaque mot : « Pour chaque client, écris *Bonjour*, puis son nom, puis une virgule, puis. . . ».

> ! On peut tout faire : calculs, décisions complexes, plusieurs fichiers, appels à d'autres bibliothèques.

Dans les deux cas, la **liste des destinataires** est le **modèle** ; seule la façon de produire le texte change.

Programmation ou template : le même résultat

→ Deux écritures du même morceau de générateur



① *le texte à produire est* ***dans des chaînes*** *du programme ; ② le texte est écrit* ***en clair****, seules les parties variables sont entre crochets.*

Comparaison des deux approches

→ D'après le tutoriel, complété

	Acceleo (template)	**Programmation (Java + API EMF)**
Style	Déclaratif : on décrit le texte	Impératif : on décrit les **actions**
Langage	Template `.mtl` + expressions OCL	Java classique (ou Python avec PyEcore)
Où est le texte produit ?	En clair, avec des trous `[.../]`	Dans des chaînes `"..."` + concaténations
Navigation	`cls.eAttributes`	`cls.getEAttributes()`
Zones protégées, multi-fichiers	Intégrées (`[protected]`, `[file]`)	À programmer soi-même
Points forts	Lisible, rapide à écrire	Flexible, débogueur Java, toute bibliothèque Java

Question flash

?

Le tutoriel dit que la programmation est « flexible mais plus détaillée ». Donnez **un exemple** de ce qui est plus détaillé en Java qu'en Acceleo.

Question flash

?

Le tutoriel dit que la programmation est « flexible mais plus détaillée ». Donnez **un exemple** de ce qui est plus détaillé en Java qu'en Acceleo.

Réponse

Le **texte** à produire : en Java il faut écrire chaque morceau entre guillemets, échapper les guillemets (\"), ajouter les retours à la ligne `\n` et gérer soi-même l'ouverture et la fermeture du fichier. En Acceleo, le texte est écrit tel quel dans le template.

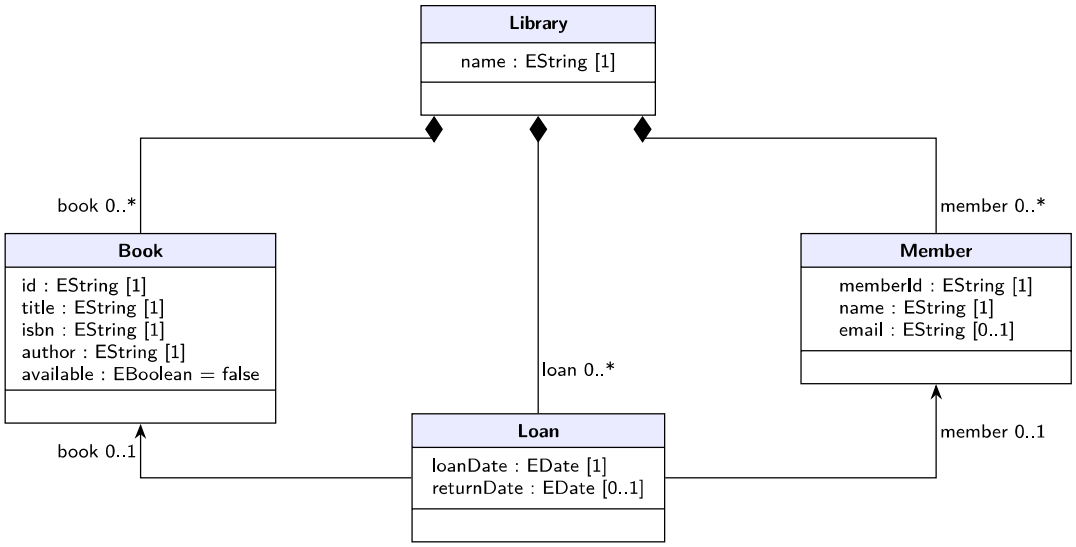
Lire un modèle avec EMF

Comment charger et parcourir un modèle Ecore en Java ?

Le modèle d'entrée : Library.ecore

→ Le fil rouge du cours, du TD et du TP (le même qu'en Lecture 3)

CIM | **PIM** | PSM | Code

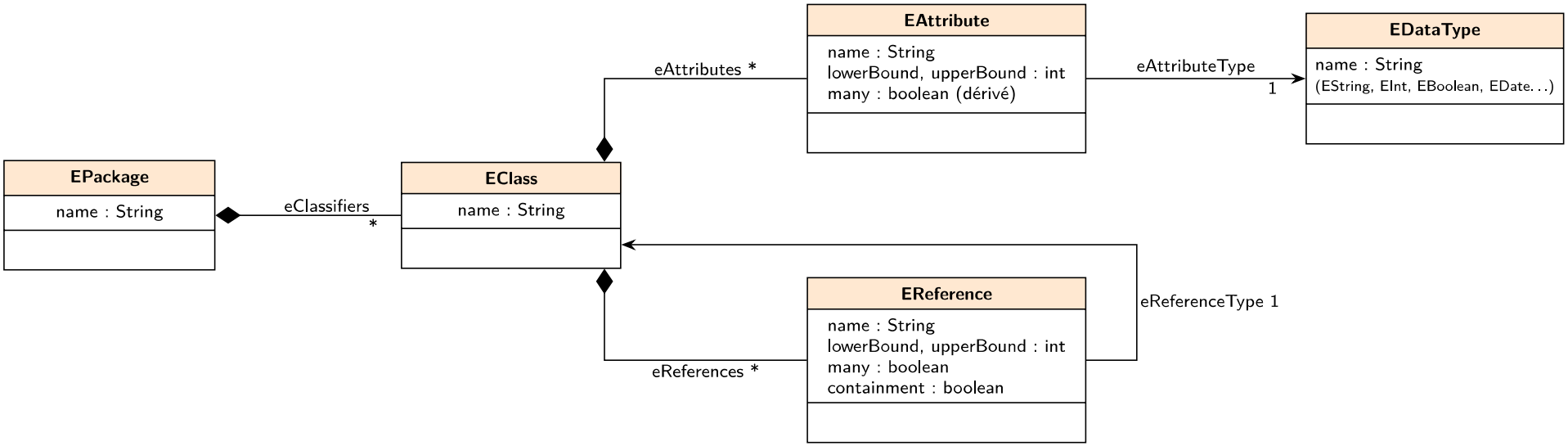


- **Library** : un nom ; contient des livres, des membres et des emprunts (**compositions** ♦).
- **Book** : id, titre, ISBN, auteur, disponibilité.
- **Member** : identifiant, nom, e-mail (facultatif `[0..1]`).
- **Loan** : dates ; référence **un** livre et **un** membre (`0..1`).

> ! Le modèle d'entrée est **un fichier** `.ecore` dessiné avec Ecore Tools.

Rappel : le méta-modèle Ecore (simplifié)

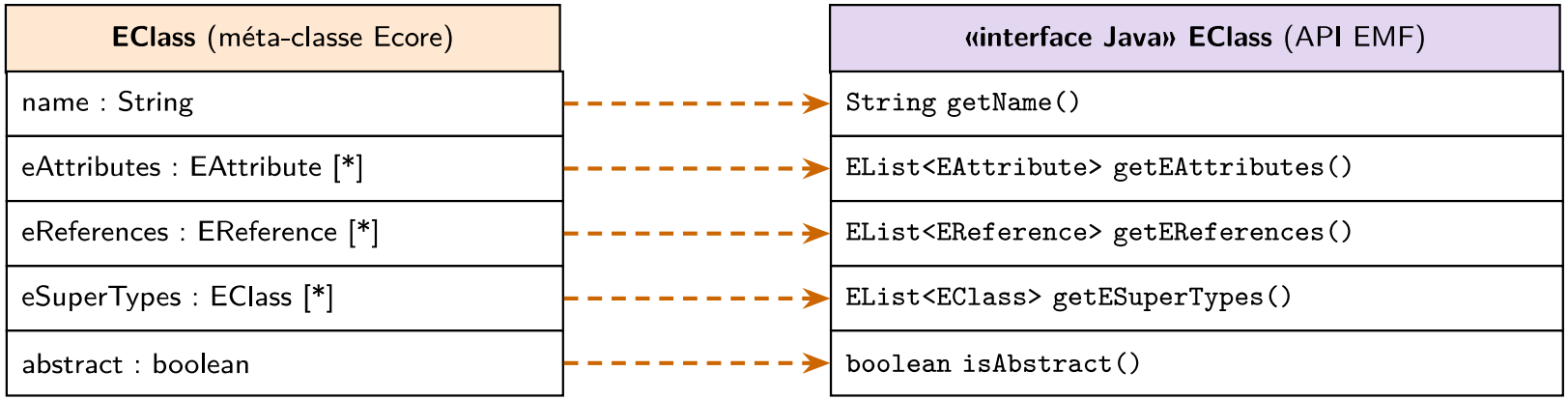
→ Les seuls concepts dont notre programme a besoin



Ecore simplifié : les seuls concepts dont on a besoin pour écrire nos templates.
`eClassifiers` contient aussi des types de données : on filtre les classes avec `->filter(EClass)`.

Des concepts Ecore aux interfaces Java

→ EMF fournit une interface Java pour chaque concept d'Ecore



Règle : chaque propriété `xxx` d'un concept Ecore se lit avec `getXxx()` (`isXxx()` pour un booléen).
Il en va de même pour `EAttribute` (`getLowerBound()`, `getEAttributeType()`...) et `EReference` (`isMany()`, `isContainment()`...).

! Si vous savez lire le méta-modèle Ecore, vous connaissez déjà l'API : il suffit d'ajouter `get` (ou `is`).

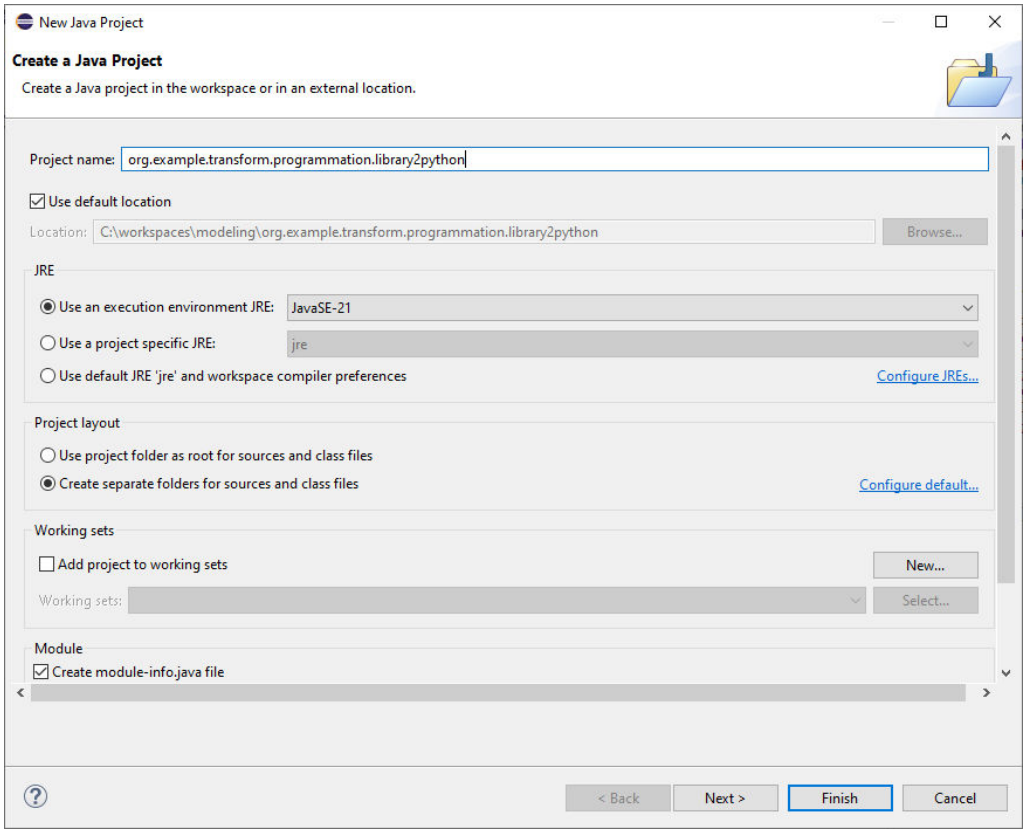
Les méthodes de l'API à connaître

→ Avec leur valeur sur library.ecore

Sur un objet...	Méthode	Renvoie	Exemple (library.ecore)
`EPackage`	`getName()`	`String`	`"library"`
`EPackage`	`getEClassifiers()`	`EList<EClassifier>`	Library, Book, Member, Loan
`EClass`	`getEAttributes()`	`EList<EAttribute>`	Book : id, title, isbn, author, available
`EClass`	`getEReferences()`	`EList<EReference>`	Loan : book, member
`EAttribute`	`getEAttributeType().getName()`	`String`	`"EString"`, `"EBoolean"`, `"EDate"`
`EAttribute`	`getLowerBound()`	`int`	email : 0 ; name : 1
`EReference`	`isMany()`	`boolean`	Library.book : true ; Loan.book : false
`EReference`	`isContainment()`	`boolean`	Library.book : true ; Loan.book : false
`EReference`	`getEReferenceType()`	`EClass`	Loan.book → Book

Créer le projet Java

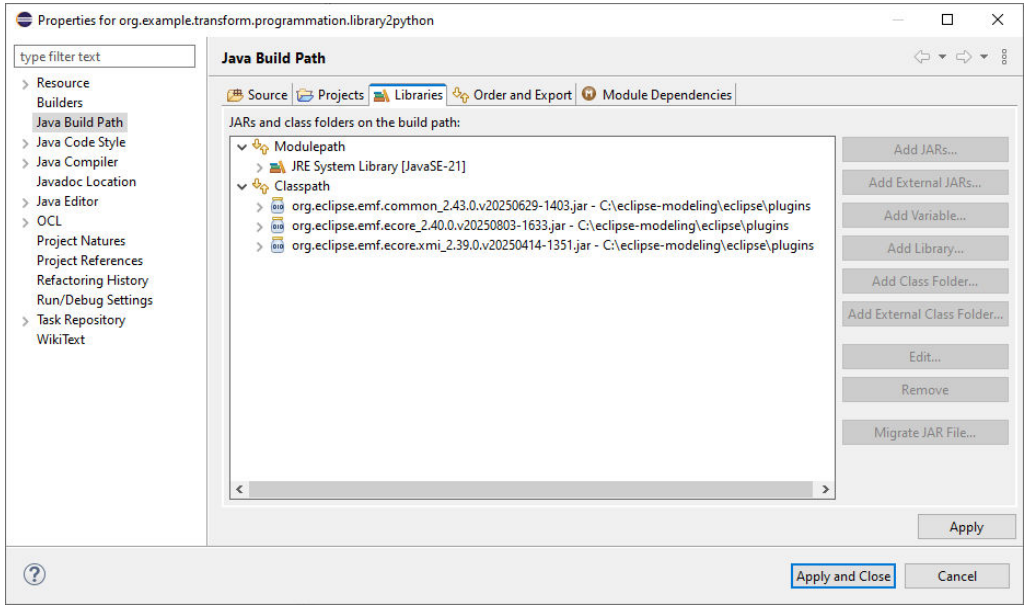
→ File → New → Java Project (tutoriel, § 1.2)



- Nom du projet : `org.example.transform.programmation.library2python`.
- JRE : **JavaSE-17** ou plus récent.
- **Décocher** « Create module-info.java file » (ou supprimer ce fichier ensuite).
- Créer le paquetage `org.example.transform` et la classe `Library2Python`.
- Créer un dossier `model/` et y copier `library.ecore`.

Ajouter les bibliothèques EMF

→ Build Path → Configure Build Path → Libraries → Classpath



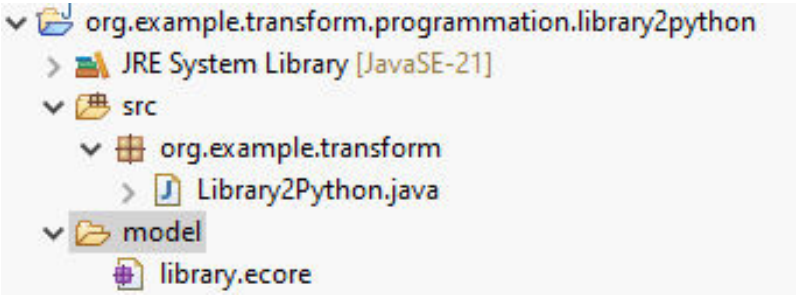
- **Add External JARs...** puis aller dans `<ECLIPSE_HOME>/plugins/`.
- Sélectionner les **trois** JAR :
  - `org.eclipse.emf.common_*.jar`
  - `org.eclipse.emf.ecore_*.jar`
  - `org.eclipse.emf.ecore.xmi_*.jar`
- Les ajouter dans **Classpath** (pas Modulepath).
- **Apply and Close**.

Sans ces JAR

Eclipse souligne en rouge `import org.eclipse.emf...` : « The import org.eclipse cannot be resolved ».

Structure du projet

→ Où sont les fichiers ?

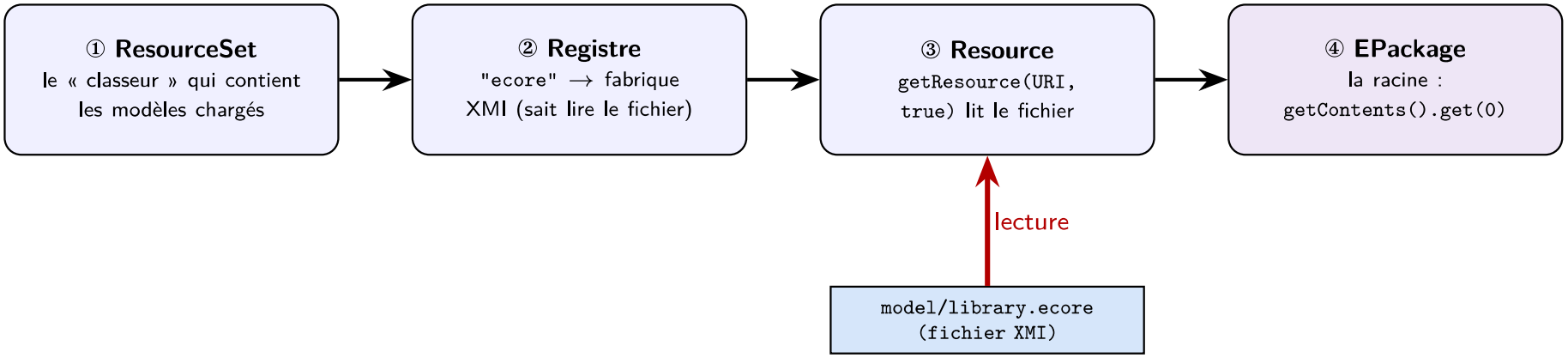


- `src/` : le **programme** de transformation (`Library2Python.java`).
- `model/` : le **modèle** d'entrée (`library.ecore`).
- `gen/` : le **code généré** (créé par le programme ; faire **F5** pour le voir).

! Ne jamais modifier à la main un fichier de `gen/` : il est **écrasé** à chaque exécution.

Charger le modèle en 4 étapes

→ Du fichier .ecore à l'objet EPackage



*En dehors d'Eclipse (programme `main`), il faut **enregistrer** la fabrique pour l'extension `.ecore` (étape ②) ; sinon : « Cannot create a resource... ».*

- La **fabrique** sait transformer le fichier XMI en objets Java ; le **registre** associe l'extension `ecore` à cette fabrique.

Le code de chargement

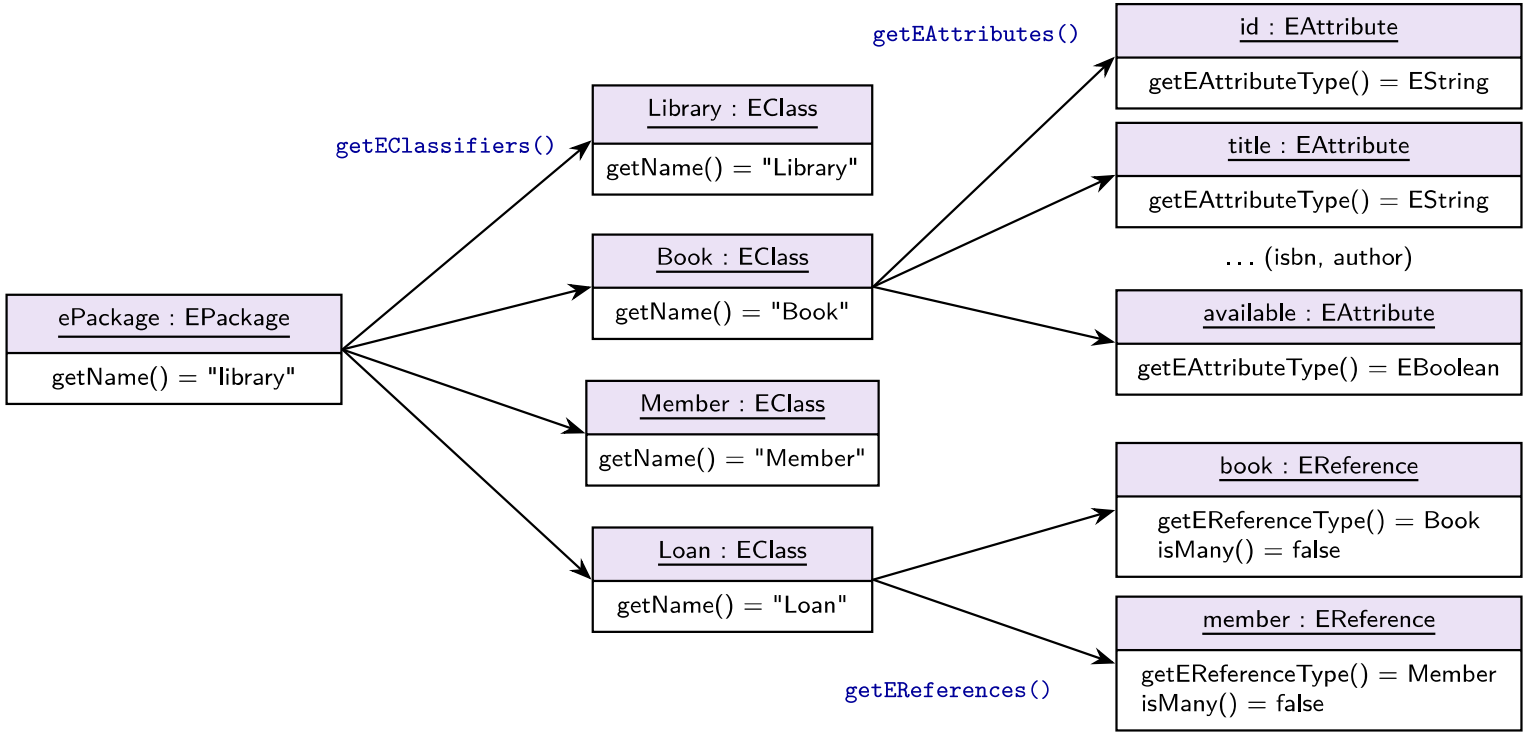
→ Library2Python.java – la méthode main (tutoriel, § 2.1)

```java
public static void main(String[] args) throws IOException {
    // 1. Créer un ResourceSet et associer l'extension .ecore à une fabrique XMI
    ResourceSet resourceSet = new ResourceSetImpl();
    resourceSet.getResourceFactoryRegistry()
               .getExtensionToFactoryMap().put("ecore", new XMIResourceFactoryImpl());
    // 2. Charger le modèle Ecore
    Resource resource = resourceSet.getResource(
            URI.createFileURI("model/library.ecore"), true);
    // 3. Obtenir le package racine
    EPackage ePackage = (EPackage) resource.getContents().get(0);
    // 4. Lancer la génération
    new Library2Python().generatePython(ePackage, "gen");
}
```

```java
import org.eclipse.emf.common.util.URI;
import org.eclipse.emf.ecore.*;                    // EPackage, EClass, EAttribute...
import org.eclipse.emf.ecore.resource.*;           // ResourceSet, Resource
import org.eclipse.emf.ecore.resource.impl.ResourceSetImpl;
import org.eclipse.emf.ecore.xmi.impl.XMIResourceFactoryImpl;
```

Ce que « voit » le programme

→ library.ecore chargé = un graphe d'objets Java



*Après chargement, le programme manipule un **graphe d'objets Java** ; il le parcourt en appelant les **getters** de l'API EMF.*

Explorer le modèle avant de générer

→ Afficher le modèle dans la console : le « Hello world » d'EMF

```java
for (EClassifier classifier : ePackage.getEClassifiers()) {
    if (classifier instanceof EClass cls) {
        System.out.println("Classe " + cls.getName());
        for (EAttribute attr : cls.getEAttributes())
            System.out.println("  - attribut " + attr.getName() + " : "
                + attr.getEAttributeType().getName()
                + " [" + attr.getLowerBound() + ".." + attr.getUpperBound() + "]");
        for (EReference ref : cls.getEReferences())
            System.out.println("  - référence " + ref.getName() + " -> "
                + ref.getEReferenceType().getName()
                + (ref.isMany() ? " (plusieurs)" : " (une seule)"));
    }
}
```

Console

```
Classe Library
  - attribut name : EString [1..1]
  - référence book -> Book (plusieurs)
  - référence member -> Member (plusieurs)
  - référence loan -> Loan (plusieurs)
Classe Book
  - attribut id : EString [1..1]
  - attribut title : EString [1..1]
  ...
Classe Loan
  - attribut loanDate : EDate [1..1]
  - attribut returnDate : EDate [0..1]
  - référence book -> Book (une seule)
  - référence member -> Member (une seule)
```



Commencez **toujours** par afficher le modèle : vous vérifiez que le chargement fonctionne et vous voyez ce que vous allez parcourir.

Question flash

? Comment obtenir, en Java, le **nom du type** de l'attribut `available` (réponse attendue : `EBoolean`) ? Et comment savoir si la référence `Loan.member` est **multiple** ?

Question flash

? Comment obtenir, en Java, le **nom du type** de l'attribut `available` (réponse attendue : `EBoolean`) ? Et comment savoir si la référence `Loan.member` est **multiple** ?

Réponse

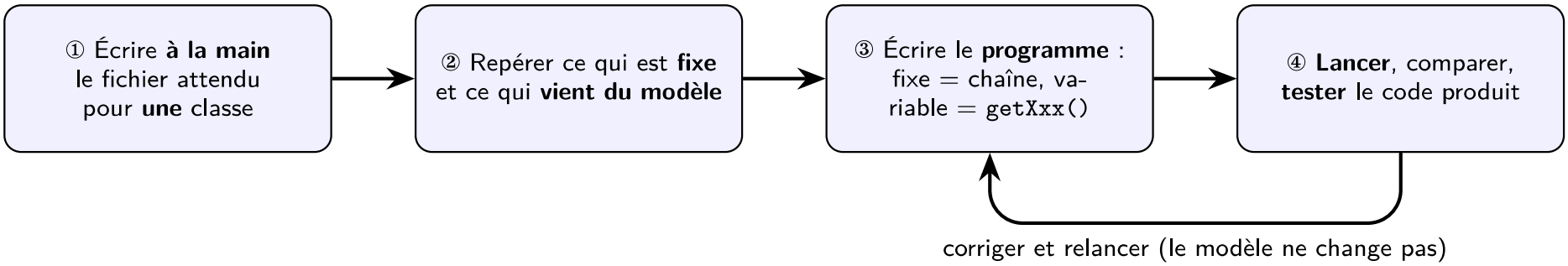
`attr.getEAttributeType().getName()` renvoie `"EBoolean"`. Pour la référence : `ref.isMany()` renvoie `false` (borne supérieure 1). Règle : propriété Ecore `xxx` → méthode `getXxx()` ou `isXxx()`.

Le générateur pas à pas

Comment écrire, étape par étape, un programme qui génère du code ?

Méthode : « l'exemple d'abord »

→ Ne jamais écrire un générateur sans savoir ce qu'il doit produire



Le fichier visé (pour Book)

```
class Book:
    def __init__(self, id=None, title=None, isbn=None, author=None, available=None
        ):
        self.id = id
        self.title = title
        self.isbn = isbn
        self.author = author
        self.available = available

    def __repr__(self):
        return f"Book(id={self.id!r}, title={self.title!r}, ...)"
```

Fixe ou variable ?

- **Fixe** : `class`, `def __init__(self`, `=None`, `self.`, `__repr__`...
- **Variable** (vient du modèle) : le nom de la classe `Book`, les noms des attributs `id`, `title`...
- Le fixe devient une **chaîne Java**, le variable un **appel** `getName()`.

Étape 1 : ouvrir le fichier et écrire l'en-tête

→ Rien n'est encore lu dans le modèle (sauf son nom)

```java
public void generatePython(EPackage ePackage, String outDir) throws IOException {
    Files.createDirectories(Paths.get(outDir));          // crée gen/ si besoin
    String fileName = outDir + "/" + ePackage.getName() + "_model.py";

    try (Writer writer = new FileWriter(fileName, StandardCharsets.UTF_8)) {
        writer.write("# -*- coding: utf-8 -*-\n");
        writer.write("# Code Python généré automatiquement à partir du modèle "
                     + ePackage.getName() + ".ecore\n");
    }                                                    // fermeture automatique
    System.out.println("Fichier " + fileName + " généré avec succès !");
}
```

Résultat : gen/library_model.py

```python
# -*- coding: utf-8 -*-
# Code Python généré automatiquement à partir du modèle library.ecore
```

- `try ( ... )` **ferme** le fichier automatiquement, même en cas d'erreur ; `StandardCharsets.UTF_8` : les accents sont bien écrits.

Étape 2 : une classe Python par EClass

→ Première boucle sur le modèle


```
for (EClassifier classifier : ePackage.getEClassifiers()) {
    if (classifier instanceof EClass cls) {     // ne garder que les classes
        writer.write("\n\nclass " + cls.getName() + ":\n");
        writer.write("    pass\n");
    }
}
```

- `getEClassifiers()` : les classes **et** les types de données du paquetage.
- `instanceof EClass cls` : on ne garde que les **classes** et on les nomme `cls` (Java 16+).
- `cls.getName()` : le **nom** de la classe courante.

Résultat

```
class Library:
    pass


class Book:
    pass


class Member:
    pass

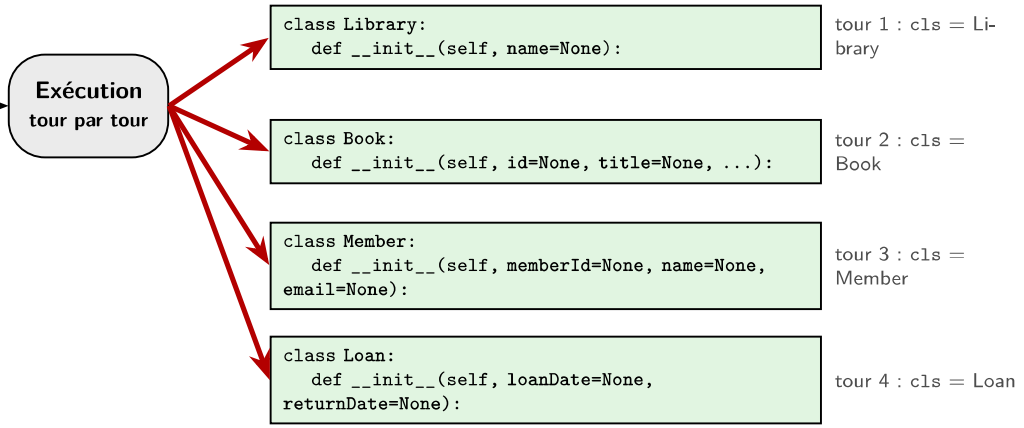

class Loan:
    pass
```

Deux boucles imbriquées

→ Ce qui se passe à l'exécution

Deux boucles imbriquées

```
for (EClassifier c : ePackage.getEClassifiers())
 if (c instanceof EClass cls) {                          // filtre
  writer.write("class " + cls.getName() + ":\n");
  for (EAttribute a : cls.getEAttributes())
   writer.write(", " + a.getName() + "=None");
 }
```



*La boucle externe parcourt les **classes** ; pour chacune, la boucle interne parcourt ses **attributs**.*
*Ajouter une classe ou un attribut au modèle ⇒ une ligne de plus dans le résultat, **sans toucher au programme**.*

Étape 3 : le constructeur __init__

→ Une méthode genClass par EClass


```
private void genClass(EClass cls, Writer writer) throws IOException {
    writer.write("class " + cls.getName() + ":\n");
    // Constructeur : un paramètre (valeur par défaut None) par EAttribute
    writer.write("    def __init__(self");
    for (EAttribute attr : cls.getEAttributes()) {
        writer.write(", " + attr.getName() + "=None");
    }
    writer.write("):\n");
    for (EAttribute attr : cls.getEAttributes()) {
        writer.write("        self." + attr.getName() + " = " + attr.getName() + "\n");
    }
}
```

Résultat pour Book

```
class Book:
    def __init__(self, id=None, title=None, isbn=None, author=None, available=None):
        self.id = id
        self.title = title
        ...
```

Étape 4 : les références

→ Une liste si la référence est multiple, None sinon


```
// EReference : liste vide si multiple (0..*), None si simple (0..1)
for (EReference ref : cls.getEReferences()) {
    writer.write("        self." + ref.getName() + " = "
                 + (ref.isMany() ? "[]" : "None") + "\n");
}
// Classe sans attribut ni référence : le corps ne doit pas être vide
if (cls.getEAttributes().isEmpty() && cls.getEReferences().isEmpty()) {
    writer.write("        pass\n");
}
```

Correction du tutoriel

Le tutoriel écrit `self.ref = []` pour **toutes** les références. Or `Loan.book` est `0..1` : c'est **un** livre, pas une liste. On teste donc `ref.isMany()`.

Résultat

```
class Library:
    def __init__(self, name=None):
        self.name = name
        self.book = []
        self.member = []
        self.loan = []

class Loan:
    def __init__(self, loanDate=None, returnDate=
      None):
        self.loanDate = loanDate
        self.returnDate = returnDate
        self.book = None
        self.member = None
```

Étape 5 : ___repr___ et le problème des séparateurs

→ Mettre une virgule ENTRE les éléments, pas après le dernier

```
writer.write("\n    def __repr__(self):\n");
writer.write("        return f\"" + cls.getName() + "(" + reprArgs(cls) + ")\"\n");

/** "id={self.id!r}, title={self.title!r}, ..." */
private String reprArgs(EClass cls) {
    List<String> parts = new ArrayList<>();
    for (EAttribute attr : cls.getEAttributes()) {
        parts.add(attr.getName() + "={self." + attr.getName() + "!r}");
    }
    return String.join(", ", parts);       // virgule ENTRE les éléments
}
```

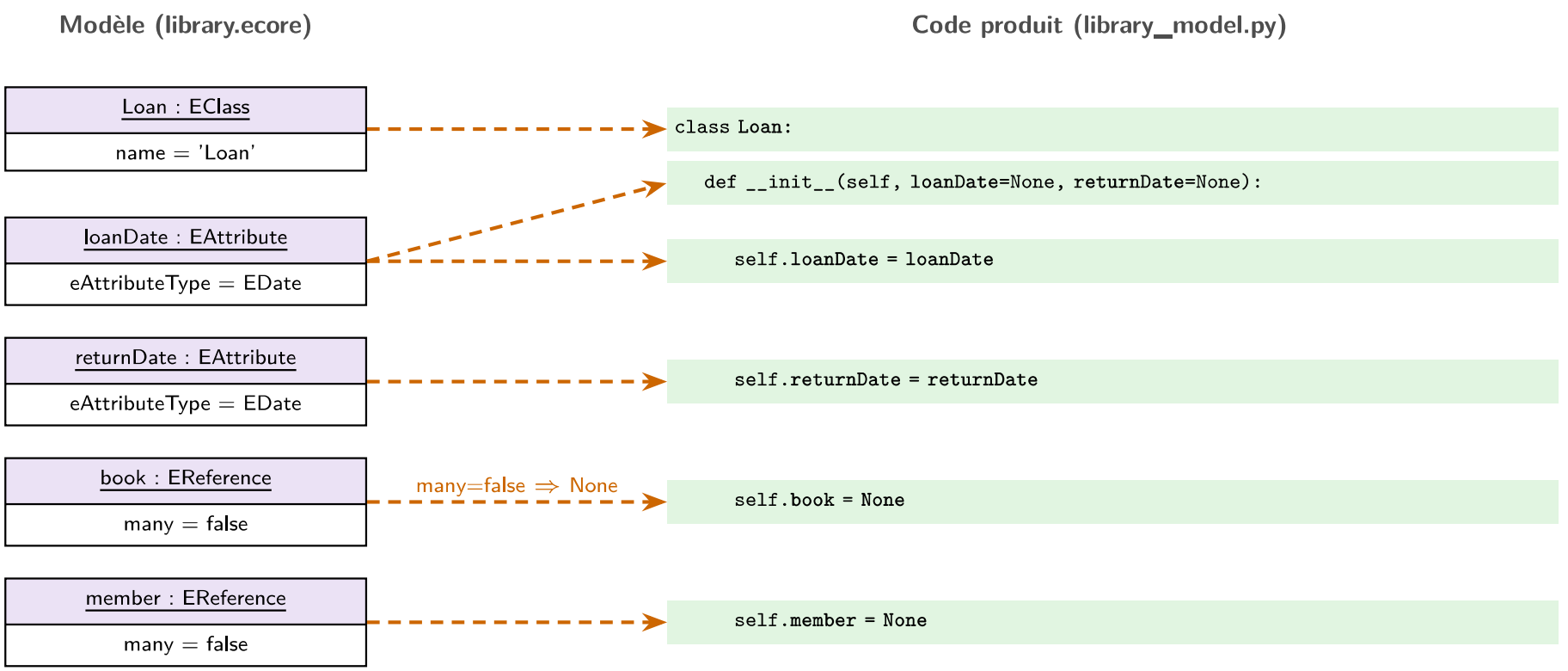
Résultat pour Member

```
    def __repr__(self):
        return f"Member(memberId={self.memberId!r}, name={self.name!r}, email={self.email!r})"
```

La traçabilité modèle → code

→ D'où vient chaque ligne produite ?

CIM | PIM | PSM | **Code**



*Chaque élément du modèle produit une (ou plusieurs) lignes : c'est la **traçabilité** modèle → code.*

Lancer et tester

→ Run As → Java Application

- Clic droit sur `Library2Python.java` → **Run As** → **Java Application**.
- La console affiche : `Fichier gen/library_model.py généré avec succès !`
- Clic droit sur le projet → **Refresh** (F5) : le dossier gen/ apparaît.
- Le code généré doit **fonctionner** : on le teste comme n'importe quel code.

```
>>> from library_model import *
>>> b1 = Book("B1", "Le Petit Prince", "978-2070612758", "Saint-Exupéry", True)
>>> loan = Loan("2026-11-02"); loan.book = b1
>>> loan
Loan(loanDate='2026-11-02', returnDate=None)
>>> loan.book.title
'Le Petit Prince'
```

! Un générateur est terminé quand le **code produit** compile et passe ses tests, pas seulement quand le programme Java s'exécute.

Trois façons d'écrire le texte

→ Choisir selon la taille du texte

Technique	Principe	Quand l'utiliser ?
`writer.write("...")`	On écrit directement dans le fichier, morceau par morceau (tutoriel).	Petits générateurs, un seul fichier.
`StringBuilder`	On construit le texte **en mémoire**, puis `Files.writeString(...)`.	Quand il faut le texte complet avant d'écrire (tests, zones protégées).
Bloc de texte `"""..."""`	Un morceau de texte en clair avec des %s remplis par `.formatted(...)`.	Gros blocs fixes : se rapproche d'un template.

```
String genStub(EClass cls) {
    return """
        class %s:
            pass
        """.formatted(cls.getName());       // bloc de texte (Java 15+)
}
```

Les pièges de l'écriture de texte

→ Les erreurs que l'on fait tous au début

Symptôme	Cause	Solution
Tout est sur une seule ligne	Oubli de `\n`	Terminer chaque ligne par `\n`
Fichier vide ou incomplet	Fichier jamais fermé (`close()` oublié)	`try (Writer w = ...) { ... }`
Accents illisibles (Ã©)	Encodage par défaut du système	`new FileWriter(nom, StandardCharsets.UTF_8)`
`(self, , id=None)` ou virgule finale	Séparateur mal placé	`String.join(", ", liste)`
Erreur Python sur une classe vide	Corps de classe vide	Écrire `pass` si rien d'autre
`FileNotFoundException`	Mauvais chemin ou **casse** (`Library.ecore` ≠ `library.ecore` sous Linux/macOS)	Vérifier le nom exact du fichier

Question flash

?

On ajoute au modèle une classe `Author` (attributs `name`, `country`) et on relance le programme **sans le modifier**. Que contient `library_model.py` ? Pourquoi ?

Question flash

?

On ajoute au modèle une classe `Author` (attributs `name`, `country`) et on relance le programme **sans le modifier**. Que contient `library_model.py` ? Pourquoi ?

Réponse

Une **classe Python de plus** : `class Author:` avec `def __init__(self, name=None, country=None):`. Le programme parcourt `getEClassifiers()` : il trouve la nouvelle classe tout seul. On change le **modèle**, pas le générateur.

Cas complet : Python, SQL, Java

Comment générer Python, SQL et Java à partir de Library.ecore ?

Le plan de travail

→ Un modèle, trois générateurs (le tutoriel et ses deux exercices)

Library2Python

Classes Python avec `__init__` et `__repr__`.

Fait dans la partie précédente.

Library2SQL

Une table par classe, clés **primaires** et **étrangères**.

Exercice 2 du tutoriel.

Library2Java

Une classe Java **par fichier**, getters/setters, `toString()`.

Exercice 1 du tutoriel.



Le chargement du modèle est **le même** pour les trois : on le range dans une classe commune `EcoreLoader`.

Factoriser le chargement : EcoreLoader

→ Écrire une fois, utiliser partout

```java
public final class EcoreLoader {
    private EcoreLoader() { }

    public static EPackage load(String path) {
        ResourceSet resourceSet = new ResourceSetImpl();
        resourceSet.getResourceFactoryRegistry()
                   .getExtensionToFactoryMap().put("ecore", new XMIResourceFactoryImpl());
        Resource resource = resourceSet.getResource(URI.createFileURI(path), true);
        return (EPackage) resource.getContents().get(0);
    }
}
```

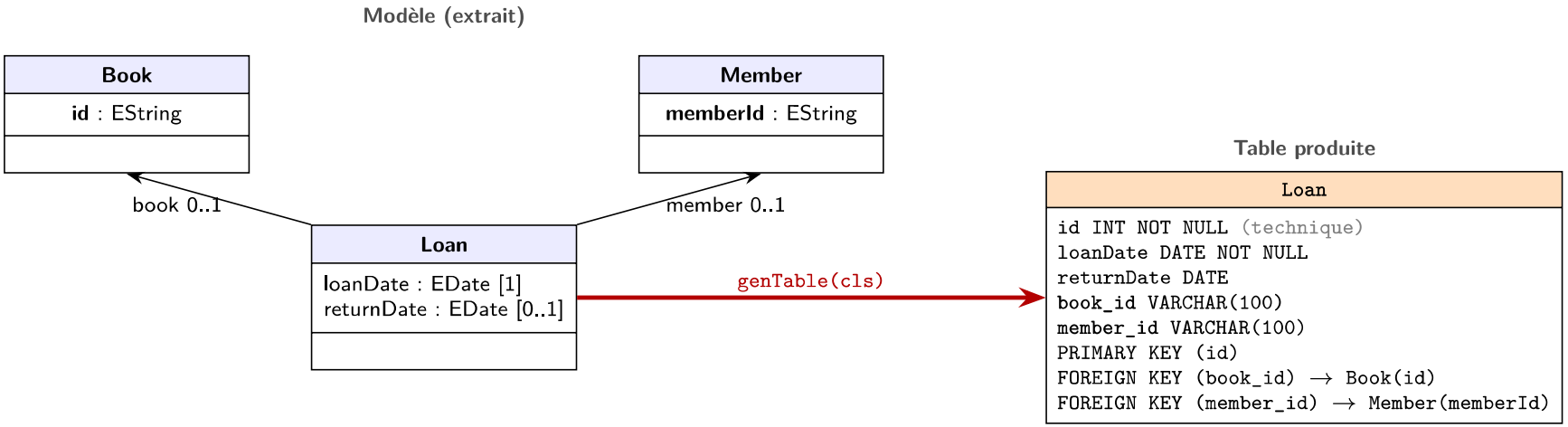
Utilisation dans Library2SQL (même chose dans Library2Java)

```java
public static void main(String[] args) throws IOException {
    String modelPath = (args.length > 0) ? args[0] : "model/library.ecore";
    EPackage ePackage = EcoreLoader.load(modelPath);
    new Library2SQL().generateSQL(ePackage, "gen");
}
```

- Le chemin du modèle peut être passé en **argument** : le même programme sert pour `school.ecore`.

SQL : les règles de traduction

→ Comment une classe devient une table



① pas d'attribut `id`/`...Id` ⇒ colonne technique `id INT` ② `lowerBound = 1` ⇒ `NOT NULL`
③ référence simple **non contenante** ⇒ colonne `<nom>_id` + `FOREIGN KEY` vers la clé primaire de la classe cible

SQL : la méthode genTable

→ Library2SQL.java – une table par EClass


```
private void genTable(EClass cls, Writer writer) throws IOException {
    writer.write("CREATE TABLE " + cls.getName() + " (\n");
    if (pkAttribute(cls) == null) {                        // pas d'identifiant
        writer.write("    id INT NOT NULL,\n");
    }
    for (EAttribute attr : cls.getEAttributes()) {         // une colonne par attribut
        String notNull = (attr.getLowerBound() > 0) ? " NOT NULL" : "";
        writer.write("    " + attr.getName() + " "
            + toSQLType(attr.getEAttributeType().getName()) + notNull + ",\n");
    }
    for (EReference ref : foreignKeys(cls)) {              // une colonne par clé étrangère
        writer.write("    " + ref.getName() + "_id " + pkType(ref.getEReferenceType()) + ",\n");
    }
    writer.write("    PRIMARY KEY (" + pkName(cls) + ")");
    for (EReference ref : foreignKeys(cls)) {
        writer.write(",\n    FOREIGN KEY (" + ref.getName() + "_id) REFERENCES "
            + ref.getEReferenceType().getName() + "(" + pkName(ref.getEReferenceType()) + ")");
    }
    writer.write("\n);\n");
}
```

SQL : les méthodes utilitaires

→ Toute la « connaissance SQL » est rangée ici

```java
static String toSQLType(String typeName) {
    switch (typeName) {
        case "EString":  return "VARCHAR(100)";
        case "EInt":     return "INT";
        case "EBoolean": return "BOOLEAN";
        case "EDate":    return "DATE";
        case "EDouble":  return "DECIMAL(10,2)";
        default:         return "VARCHAR(255)";
    }
}
/** Convention : un attribut nommé id ou se terminant par Id est la clé primaire. */
static boolean isPK(EAttribute attr) {
    return attr.getName().equals("id") || attr.getName().endsWith("Id");
}
/** Références simples (0..1 ou 1) et non contenantes = clés étrangères. */
static List<EReference> foreignKeys(EClass cls) {
    List<EReference> result = new ArrayList<>();
    for (EReference ref : cls.getEReferences())
        if (!ref.isContainment() && !ref.isMany()) result.add(ref);
    return result;
}
```

Avec `pkAttribute(cls)` (premier attribut clé ou `null`), `pkName(cls)` et `pkType(cls)` : voir le TP.

SQL : le résultat

→ Extrait de gen/library.sql (exécuté dans SQLite)

```sql
CREATE TABLE Book (
    id VARCHAR(100) NOT NULL,
    title VARCHAR(100) NOT NULL,
    isbn VARCHAR(100) NOT NULL,
    author VARCHAR(100) NOT NULL,
    available BOOLEAN NOT NULL,
    PRIMARY KEY (id)
);

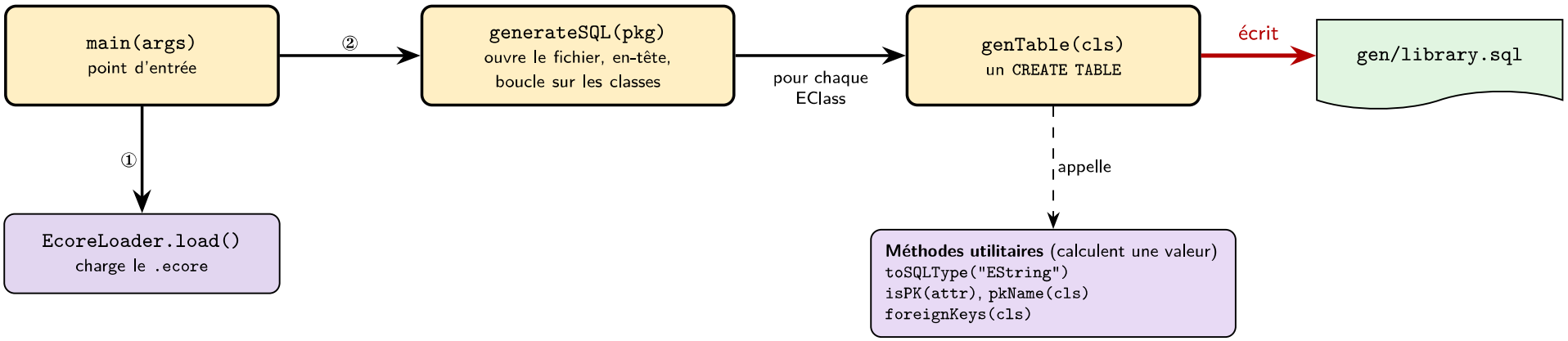
CREATE TABLE Loan (
    id INT NOT NULL,
    loanDate DATE NOT NULL,
    returnDate DATE,
    book_id VARCHAR(100),
    member_id VARCHAR(100),
    PRIMARY KEY (id),
    FOREIGN KEY (book_id) REFERENCES Book(id),
    FOREIGN KEY (member_id) REFERENCES Member(memberId)
);
```

- Une table par classe : **4 tables**.
- `Book` a un identifiant `id` : c'est la clé primaire.
- `Loan` n'en a pas : colonne technique `id INT`.
- Les références `book` et `member` deviennent des **clés étrangères**.
- Les compositions de `Library` (listes) ne donnent **pas** de colonne.

! Après quelques `INSERT`, la jointure `Loan–Book–Member` renvoie bien « Sara, Le Petit Prince ».

Organisation d'un générateur

→ Une méthode par concept à produire + des méthodes utilitaires



*Comme en Acceleo (templates + queries) : une méthode **par concept** à produire, et des méthodes **utilitaires** qui ne produisent pas de texte mais calculent une valeur.*

Java : un fichier par classe

→ Library2Java.java – la méthode principale

```
public void generateJava(EPackage ePackage, String outDir) throws IOException {
    Path dir = Paths.get(outDir, ePackage.getName());        // gen/library/
    Files.createDirectories(dir);
    for (EClassifier classifier : ePackage.getEClassifiers()) {
        if (classifier instanceof EClass cls) {
            Path file = dir.resolve(cls.getName() + ".java");   // un fichier par classe
            String userCode = readUserCode(file, cls.getName());
            Files.writeString(file, genClass(cls, ePackage, userCode),
                              StandardCharsets.UTF_8);
        }
    }
}
```

- `Paths.get(outDir, ePackage.getName())` : le dossier porte le nom du **paquetage** (`gen/library/`), comme le package Java.
- Un tour de boucle = **un fichier** : `Library.java`, `Book.java`, `Member.java`, `Loan.java`.

Java : genClass (1/2)

CIM | PIM | PSM | **Code**

→ Construire le texte dans un StringBuilder : en-tête, champs, constructeur

```
StringBuilder sb = new StringBuilder();
sb.append("package ").append(ePackage.getName()).append(";\n\n");
sb.append("public class ").append(cls.getName()).append(" {\n\n");
for (EAttribute attr : cls.getEAttributes()) {                  // champs
    sb.append("    private ").append(attrType(attr)).append(" ")
      .append(attr.getName()).append(";\n");
}
for (EReference ref : cls.getEReferences()) {
    sb.append("    private ").append(refType(ref)).append(" ").append(ref.getName());
    if (ref.isMany()) sb.append(" = new ArrayList<>()");
    sb.append(";\n");
}
List<String> params = new ArrayList<>();                        // constructeur complet
for (EAttribute attr : cls.getEAttributes()) params.add(attrType(attr) + " " + attr.getName());
sb.append("    public ").append(cls.getName())
  .append("(").append(String.join(", ", params)).append(") {\n");
```

Java : genClass (2/2)

→ Getters et setters de chaque attribut


```
for (EAttribute attr : cls.getEAttributes()) {
    String n = attr.getName();
    sb.append("\n    public ").append(attrType(attr)).append(" ")
      .append(getterName(attr)).append("() {\n");               // getId() / isAvailable()
    sb.append("        return this.").append(n).append(";\n    }\n");
    sb.append("\n    public void set").append(upperFirst(n)).append("(")
      .append(attrType(attr)).append(" ").append(n).append(") {\n");
    sb.append("        this.").append(n).append(" = ").append(n).append(";\n    }\n");
}
```

- Même principe pour les références : un getter, et un setter seulement si la référence est **simple**.
- `toString()` est construit avec `String.join(" + \", \" + ", ...)` : pas de virgule en trop.

Java : le dictionnaire des types

→ Les méthodes utilitaires Ecore → Java

```
static String toJavaType(String typeName) {
    switch (typeName) {
        case "EString":  return "String";
        case "EInt":     return "int";
        case "EBoolean": return "boolean";
        case "EDouble":  return "double";
        case "EDate":    return "Date";
        default:         return "Object";
    }
}
static String refType(EReference ref) {
    String target = ref.getEReferenceType().getName();
    return ref.isMany() ? "List<" + target + ">" : target;
}
static String getterName(EAttribute attr) {
    String prefix = attr.getEAttributeType().getName().equals("EBoolean") ? "is" :
        "get";
    return prefix + upperFirst(attr.getName());
}
```

Ecore	Java	SQL
EString	String	VARCHAR(100)
EInt	int	INT
EBoolean	boolean	BOOLEAN
EDouble	double	DECIMAL(10,2)
EDate	Date	DATE
0..* vers Book	List<Book>	–

Java : le résultat

→ gen/library/Book.java (extrait) – compile avec javac


```java
package library;
...
public class Book {

    private String id;
    ...
    private boolean available;

    public Book(String id, String title, String isbn, String author, boolean
        available) {
        this.id = id;
        ...
    }

    public boolean isAvailable() {
        return this.available;
    }
    ...
    // Start of user code methodes Book
    // Ajoutez ici votre code : il sera conservé à la prochaine génération
    // End of user code
}
```

- `available` est un `EBoolean` : getter `isAvailable()`.
- Le constructeur reçoit **tous** les attributs, dans l'ordre du modèle.
- Les 4 fichiers compilent et un petit programme de test les utilise (voir TP).

! Ces fichiers sont **identiques** à ceux produits par les templates Acceleo de la Lecture 3.

Zones protégées : garder le code écrit à la main

→ À programmer soi-même (Acceleo le fait avec [protected])

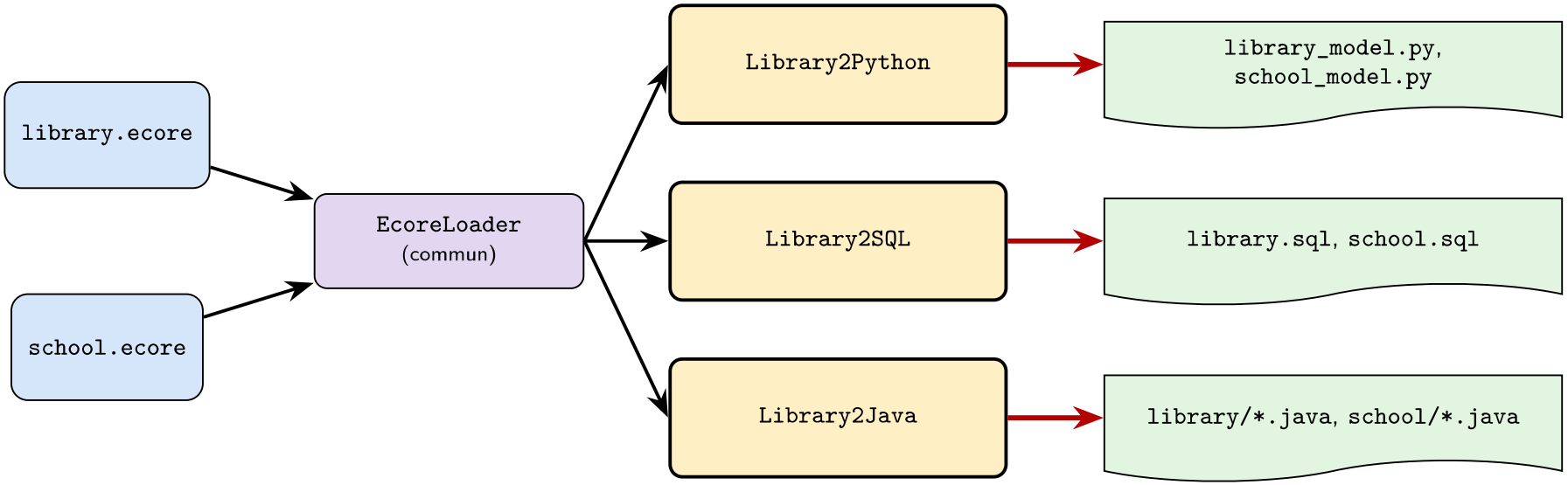


Acceleo offre `[protected]` « clé en main » ; en programmation, c'est à nous d'écrire ce mécanisme (une dizaine de lignes).

```
static String readUserCode(Path file, String className) throws IOException {
    String defaultCode = "    // Ajoutez ici votre code : il sera conservé ...\n";
    if (!Files.exists(file)) return defaultCode;                // 1re génération
    String old = Files.readString(file, StandardCharsets.UTF_8);
    String begin = "// Start of user code methodes " + className + "\n";
    int i = old.indexOf(begin);
    int j = old.indexOf("    // End of user code");
    return (i >= 0 && j > i) ? old.substring(i + begin.length(), j) : defaultCode;
}
```

Un modèle, trois cibles. . . et plusieurs modèles

→ La vraie puissance de l'approche



*Les générateurs utilisent les concepts d'**Ecore**, pas ceux de la bibliothèque : 3 générateurs × 2 modèles = 6 résultats, **sans modifier une ligne**.*

- Lancement avec un argument : **Run Configurations** → **Arguments** : `model/school.ecore`.

La même transformation en Python : PyEcore

→ L'API EMF existe aussi en Python (pip install pyecore)

```python
from pyecore.resources import ResourceSet, URI
from pyecore.ecore import EClass

rset = ResourceSet()
pkg = rset.get_resource(URI("model/library.ecore")).contents[0]   # EPackage

lines = ["# -*- coding: utf-8 -*-"]
for cls in pkg.eClassifiers:
    if not isinstance(cls, EClass):
        continue
    params = "".join(f", {a.name}=None" for a in cls.eAttributes)
    lines += ["", "", f"class {cls.name}:", f"    def __init__(self{params}):"]
    lines += [f"        self.{a.name} = {a.name}" for a in cls.eAttributes]
    lines += [f"        self.{r.name} = {'[]' if r.many else 'None'}" for r in cls.eReferences]

with open(f"gen/{pkg.name}_model.py", "w", encoding="utf-8") as f:
    f.write("\n".join(lines) + "\n")
```



Mêmes concepts (`eClassifiers`, `eAttributes`, `many`), sans les `get` : on retrouve **les noms du méta-modèle**.

Question flash

?

Dans `library.ecore`, la référence `Library.book` (0..*) ne produit **aucune** colonne SQL, alors que `Loan.book` (0..1) produit `book_id`. Quelle condition du programme fait cette différence ?

Question flash

? Dans `library.ecore`, la référence `Library.book` (0..*) ne produit **aucune** colonne SQL, alors que `Loan.book` (0..1) produit `book_id`. Quelle condition du programme fait cette différence ?

Réponse

La méthode `foreignKeys(cls)` ne garde que les références `!ref.isContainment() && !ref.isMany()`. `Library.book` est une **composition multiple** : elle est ignorée (en relationnel, c'est la table `Book` qui porterait la clé vers `Library`). `Loan.book` est simple et non contenante : elle devient une clé étrangère.

Bonnes pratiques et synthèse

Quelles erreurs éviter ? Programmation ou template ? Que retenir ?

Les erreurs les plus fréquentes

→ Message d'erreur → cause → solution

Message	Cause	Solution
`The import org.eclipse cannot be resolved`	JAR EMF absents du Classpath	Ajouter les 3 JAR (common, ecore, ecore.xmi)
`Cannot create a resource for 'file:...'`	Fabrique non enregistrée pour `ecore`	`...getExtensionToFactoryMap().put("ecore", ...)`
`FileNotFoundException`	Chemin ou casse du nom faux	Chemin relatif au **projet** : `model/library.ecore`
`ClassCastException ... cannot be cast to EPackage`	Le fichier chargé n'est pas un `.ecore` (ex. une instance `.xmi`)	Charger le bon fichier
`must be declared in a module`	`module-info.java` présent	Supprimer `module-info.java`
Rien dans `gen/`	Vue non rafraîchie	**Refresh** (F5) sur le projet

Bonnes pratiques

→ Pour des générateurs lisibles et maintenables

- **Une méthode par concept** à produire (`genClass`, `genTable`...) et des **méthodes utilitaires** pour les calculs (`toSQLType`, `isPK`...).
- **Un seul** code de chargement (`EcoreLoader`) pour tous les générateurs.
- Ne jamais écrire un nom du modèle « en dur » (`"Book"`) : toujours `cls.getName()`. Le générateur doit marcher pour **tout** `.ecore`.
- Toujours `try ( ... )`, `UTF-8` et un dossier de sortie dédié (`gen/`).
- Tester le **code produit** (compilation, exécution), sur **au moins deux** modèles.
- En-tête « Code généré – ne pas modifier » dans chaque fichier produit.

Programmation ou template : comment choisir ?

→ Les deux approches sont complémentaires

Choisir un template (Acceleo)

Le texte produit est **long** et **régulier** (classes, pages HTML...).

On veut **voir** le résultat dans le générateur.

On a besoin de zones protégées « clé en main ».

Choisir la programmation (Java/EMF)

La logique est **complexe** (calculs, tris, graphes, décisions).

On veut **intégrer** la génération dans une application (bouton, serveur, ligne de commande).

On veut aussi faire du **M2M** ou de la **validation**.

! On combine souvent : un programme Java prépare et calcule, puis appelle des templates pour écrire le texte.

Pour aller plus loin : M2M par programmation

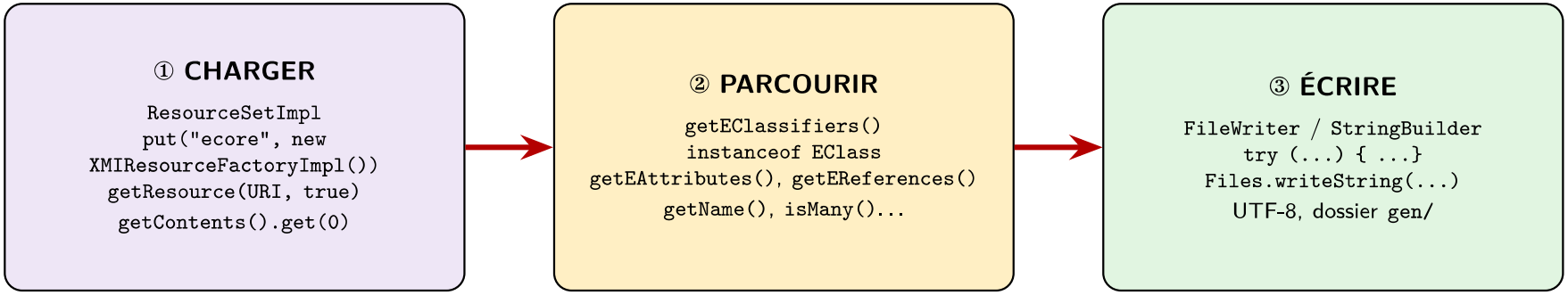
→ Le même programme peut créer un modèle au lieu d'un texte

- Créer des éléments : `EcoreFactory.eINSTANCE.createEClass()`, `createEAttribute()`...
- Les remplir : `setName("Table_Book")`, puis les ranger : `ePackage.getEClassifiers().add(...)`.
- Enregistrer : créer une `Resource` avec `resourceSet.createResource(URI)`, y ajouter la racine, puis `resource.save(null)`.
- C'est exactement ce que fait ATL... mais écrit à la main : plus long, plus libre.

> **!** Approche par programmation de MDA : **API du méta-modèle source → programme Java → API du méta-modèle cible**.

Synthèse

→ Ce qu'il faut retenir



- Une transformation par programmation **charge** le modèle, le **parcourt** avec l'API EMF et **écrit** le résultat.
- Propriété Ecore `xxx` → méthode `getXxx()` / `isXxx()`.
- Le générateur est **indépendant** du modèle : il fonctionne pour tout fichier `.ecore`.
- Même résultat qu'Acceleo, avec **plus de contrôle** et **plus de code** à écrire.

Aide-mémoire API EMF

→ À garder sous les yeux pendant le TD et le TP

Besoin	Code Java
Charger un `.ecore`	`EPackage p = EcoreLoader.load("model/library.ecore");`
Parcourir les classes	`for (EClassifier c : p.getEClassifiers()) if (c instanceof EClass cls) { ... }`
Attributs / références	`cls.getEAttributes()` / `cls.getEReferences()`
Nom, type	`attr.getName()`, `attr.getEAttributeType().getName()`
Obligatoire ? multiple ?	`attr.getLowerBound() > 0`, `ref.isMany()`
Composition ? cible ?	`ref.isContainment()`, `ref.getEReferenceType()`
Écrire un fichier	`try (Writer w = new FileWriter(nom, StandardCharsets.UTF_8)) { w.write(...); }`
Joindre avec un séparateur	`String.join(", ", liste)`

Pour s'entraîner

→ TD et TP associés

TD : Transformation par programmation

Lire l'API, **prédire** la sortie d'un programme, compléter et **corriger** des générateurs (Python, SQL, Java, Markdown).

Sur papier, avec corrigé.

TP : Tutoriel complet

Dans Eclipse : projet Java + JAR EMF, `Library2Python`, `Library2SQL`, `Library2Java`, zones protégées, `school.ecore`.

Avec squelettes, solutions et fichiers attendus.

Question de synthèse

→ Question flash

Question flash

?

Votre équipe doit générer, à partir du même modèle, (1) 40 pages HTML de documentation très régulières et (2) un rapport qui calcule pour chaque classe son nombre de dépendances et trie les classes. Quelle approche pour chacun ?

Question de synthèse

→ Question flash

Question flash

?

Votre équipe doit générer, à partir du même modèle, (1) 40 pages HTML de documentation très régulières et (2) un rapport qui calcule pour chaque classe son nombre de dépendances et trie les classes. Quelle approche pour chacun ?

Réponse

(1) Les pages HTML : **template** (Acceleo) – texte long et régulier, plus lisible. (2) Le rapport : **programmation** (Java/EMF) – calculs et tri, plus naturels en Java. On peut aussi combiner : Java calcule, un template écrit.